

Zahradnice: a Grammar Garden as an Experimental Substrate

What one stochastic rewriting loop can test, and what it costs

Jan Popelka & Claude (Anthropic) — August 2026

github.com/popojan/zahradnice branch research

Abstract. Zahradnice is a terminal game engine whose programs are weighted, stochastic 2D rewriting grammars. This companion paper reports what happened when we used it as a research instrument: within one engine, without a single engine change, we expressed and measured directional selection, quantitative-trait tracking, three kinds of ecological memory, reinforcement learning with reversal, and finally the evolution of learning itself — species differing only in learning architecture competing to a stable polymorphism. Every claim is backed by exact accounting: replaying the event trace reconstructs the final screen glyph-for-glyph (about 350 runs, zero mismatches). The centerpiece is a catalogue of a dozen textbook laws of ecology, psychology and queueing theory that *fell out of the substrate unasked*, several of them by falsifying the authors’ predictions. The headline metric is the cost of asking: the median experiment here is 25–120 generated rules and one to three hours from hypothesis to exactly-accounted verdict. We lose three to five orders of magnitude of runtime to specialized code — and argue why the trade is often right: one engine fits all programs, because what the engine implements is not any particular model but the meta-law (stochastic local rewriting) that the models share; each program is just a particular nature.

This document is self-contained. Deeper detail lives in the repository documents referenced per section (experiments//*-results.md, GRAMMAR.v2.md).*

1	The substrate in one page	1
2	Method: exact accounting, generators, composition	2
3	The experiments, briefly	3
3.1	Trait ladder: tracking as a driven oscillator	3
3.2	Seed bank: the memory triad	3
3.3	Bandit: learning without an optimizer	3
3.4	Garden: the evolution of learning	4
4	The laws that fell out unasked	4
5	The main quantity: what an experiment costs	5
6	Honest limits	6
7	Where this leaves the project	6
8	References	6

1 The substrate in one page

A Zahradnice program is a plain text file of rules. A rule names a *left-hand character* to anchor on, a trigger, a replacement, a weight, an integer score, and a small 2D body that may read and write the anchor’s neighbourhood. The screen — a toroidal character grid — *is* the entire state. Execution is a jump chain: each input character (a keypress, or a scripted byte in headless mode) is one event; the engine gathers every (rule, position) pair whose pattern currently matches, samples exactly one with probability proportional to its weight, and applies it. There is no other clock, no hidden state, no fitness function; the score register is write-only telemetry that no rule can read.

Three consequences shape everything below.

- **Rarity is relative.** A weight has no absolute meaning; an event’s probability is its share of the currently applicable mass. Slow” processes must be engineered as ratios against a persistent clock process, and quiescent states silently renormalize rare rules to certainty. We hit both edges and turned both into measurements.
- **Every event is something.** Choosing a wrong action consumes the event — opportunity cost is not modelled, it is automatic. This makes capture-rate arithmetic exact (Section 5).
- **Determinism per seed.** A seeded single-threaded run is bit-reproducible and replayable. Our verification standard throughout: an analyzer classifies every rule of the program (by its source line, which the event trace records), replays the trace into per-species populations, and the result must equal the final screen *glyph-for-glyph*. All results below meet it; the check caught about a dozen of our own bugs before they could become findings.

To make this concrete, here is an excerpt of a complete program — the two-species forest of Section 3 — exactly as the engine reads it. *This text is the program and the engine’s only first-class input; the Python generators mentioned below are authoring aids that emit files like this one, nothing more.*

```
#timing 1 3000          % lightning key fires every 3 s (interactive)
#timing T 0             % T = the ambient event trigger
^.**                   % fill the field with ground '.'
^cXX                  % a few random c plants ...
^dXX                  % ... and d plants

==.Tc20    1 6          % ground next to c sprouts c (weight 6, +1)
@c@@
==.Td60    1 1          % ground next to d sprouts d (weight 1)
@d@@
==c1A13    0 1          % lightning ignites c (equal for d)
@@@
==ATA13    0 4          % fire spreads fast into c (weight 4) ...
@c@@@A
==ATA13    0 0.25       % ... and slowly into d (weight 0.25)
@d@@@A
==dT.60    0 0.01       % d pays rent: rare natural death
@@@
==AT.00    -1 1         % burnout
@@@
```

A rule is one header line (anchor character, trigger, replacement, colours, then score and weight) plus a small body: the first @ marks the anchored cell, the second separates the read side from the write side, the third anchors the write. The body @c@@ reads a c to my east”; @c@@@A writes fire into it. Eight such rules are the whole ecology.

The environment enters through the input stream: a season” is simply which trigger characters arrive. Rules are immutable at runtime — the laws of physics; all adaptation below lives in configuration — matter. The one honest boundary note: the engine did receive two fixes during this work (UTF-8 wide-file output, locale-independent number parsing) — bug repairs, not capabilities; the capability set used everywhere is the engine as it stood.

2 Method: exact accounting, generators, composition

Generators. Programs are emitted by small Python scripts (40–150 lines); a mechanism (mutation, dormancy, punishment) is a 10–40-line generator delta. This is where the concise” in our thesis lives: we never claim a mechanism is expressible in principle — universality makes that trivial — only that it costs a handful of legible rules.

Exactness. Each experiment’s analyzer re-derives the final state from the trace and reconciles against the screen dump. This is cheap (the trace records each applied rule’s source line) and merciless: it caught misplaced deposits, stale classifiers, initial symbols not counted, and a status line polluting glyph counts. Time until you trust the number” is part of design time, and here it is amortized into the engine’s tracing.

Composition. The engine’s ordinary `#include` turns out to be an environment/agent framework: the environment is a rule library (field, action space, reward tagging), an agent is a top-level file adding its rule mass. Swapping agents cannot perturb the world — proven bit-exactly: the null agent’s runs are identical to the monolithic control’s, because never-applicable rules leave the jump chain untouched. Policies outside the mass-action family can use the deterministic prefix-replay loop instead (any external algorithm, same environment).

3 The experiments, briefly

Each experiment below ran on the stock engine, is generated, exactly-accounted, and documented in full in the linked repository file. Numbers are means over 6–8 seeds unless stated.

3.1 Trait ladder: tracking as a driven oscillator

Five plant species form a heritable ladder (growth rate traded against flammability; reproduction copies the parent’s rung with mutation to neighbours). Environment: lightning period P . The population’s mean trait tracks fire frequency *monotonically in time-average* (4.36 at $P=200$ down to 3.58 at $P=32000$, absorbing at the fast end without fire), but the tracker is a relaxation oscillator: fire filters upward instantly, succession decays downward with a measured time constant of 30–60k events, and the regime is set by P/τ . Finals are phase samples; only cycle averages are honest observables.

`experiments/soc/ladder-results.md`

3.2 Seed bank: the memory triad

Dormant seeds — cells that fire and growth simply have no rules for — are a protected archive. With *consumptive* readout (germination spends the seed) memory is faithful and bounded: retention is set by weight ratios (a slow bank’s archive survives 240k events of adversarial fire, roughly 1200 fire cycles, and still halves re-adaptation time), while influence is capped — Little’s law pins readout flux to deposit flux, so influence times retention equals bank size, independent of the germination rate. The heredity-only control retains nothing: population memory is hot storage, erased by the very selection that wrote it. With *catalytic* readout (a seed templates its species into adjacent ground without being consumed) retention becomes unbounded — the archive turns into a self-renewing lineage — at three prices: winner-take-all consensus, destabilized dynamics (a lagged copy fed back is phase-wrong under periodic driving), and one spectacular emergent: a fireproof seed consensus of the flammable pioneer keeps feeding fuel into burning gaps, and the fire becomes self-sustaining with no lightning at all — pyrogenic niche construction, the eucalyptus syndrome, uninvited.

`experiments/soc/bank-results.md`

3.3 Bandit: learning without an optimizer

Two symmetric actions bloom on ground; the input season decides which one pays. The paying rule deposits its arm’s token *in the same rewrite that scores*; tokens catalyze their arm and decay. That is the whole learner — 25 rules. It acquires the paying arm in 2k events, re-acquires after every reversal (36 of 36 blocks), captures $\pi/(2+\pi)$ of events at measured policy $\pi \approx 0.90$ (predicted 310 per thousand, measured 310; random control predicted 250, measured 251), and exhibits the economics of learning: with frequent reversals the learner nets *less* than the non-learner (reversals cost 15–19k wrong-way events each); with rare reversals it wins by 21%. Adding the missing half of Michie’s 1961 MENACE — punishment, one confiscation rule per arm: a failed advised attempt eats its adviser — makes unlearning three times faster and moves the crossover: the learner then beats the control even on the fast schedule. Taxonomically this is not ϵ -greedy,

UCB, or Thompson; it is probability matching on additively reinforced propensities with forgetting — the Roth-Erev model, a linear reward-penalty learning automaton, Herrnstein’s matching law.
experiments/bandit/bandit-results.md

3.4 Garden: the evolution of learning

Three species identical in metabolism, differing only in learning machinery — none, reward-inaction, reward-penalty — compete for ground in a seasonal world. Reward is metabolic: a correct attempt writes an offspring (learners deposit their token in the same rewrite, west, where the parent itself reads it next); a wrong attempt wastes the event; rent kills. No fitness function exists anywhere — fitness *is* the population trajectory.

When errors are cheap, fitness is attempt-mass-dominated: at two-thousand-event seasons the learners’ memories are *anti-correlated* with the world (correct-share 0.32 — always one season behind) and they still outbreed the non-learner, because tokens fuel activity. Busy beats right when errors cost only a wasted event. When errors can kill (a wrong attempt is lethal with probability ≈ 0.2), information takes over: the naive learner’s edge collapses as seasons accelerate, and punishment’s advantage *grows* with switching speed (+21/+18/+57 over the naive learner at constant/30k/2k seasons) — the value of unlearning is set by the price of error. A season-length scan pins the adaptive-value-of-learning threshold at roughly one to four thousand events per season: the naive learner falls to the non-learner’s level between 4k and 1k, while punishment stays profitable about four times deeper into fast worlds. With a mutation ladder between architectures and a start of twelve non-learners, evolution discovers learning unaided within 30k events and settles into a stable polymorphism whose shares reproduce the seeded competition quantitatively; no architecture ever fixates. Coexistence is held partly by design (immigration), partly by slow local competition, and partly by a genuine stabilizer: **memory pays rent in land** — learner territory carries its token load, which throttles its own vacancy supply and keeps the lean ignorant competitive.

experiments/garden/garden-results.md

4 The laws that fell out unasked

This section is, in our view, the strongest evidence the substrate offers. None of the following was programmed in; each emerged in the measurements, was recognized afterwards, and in several cases first *falsified* the authors’ explicit prediction. The recurring experience — surprise, then the retrospective of course” — is what a good instrument feels like.

- **Relaxation oscillation & bandwidth.** [1] The trait tracker is a driven oscillator; tracking quality is forcing period over relaxation time — a transfer function measured from ecology.
- **Competition-colonization trade-off.** [2] Fire resistance must pay rent or it dominates every regime (Tilman’s structure, rediscovered as a failed first design).
- **Seed banks are ecosystem memory.** [3,4] Dormancy decouples retention from disturbance — ecology’s own answer, complete with resurrection-ecology-grade archives.
- **Little’s law.** [5] Bank population equals deposit flux times residence time; with destructive readout, influence $\times$ retention = bank size, and the germination rate is a delay knob that cannot buy throughput.
- **Delayed feedback destabilizes.** [6] A low-pass lagged copy of the state fed back positively increases oscillation amplitude — control theory’s oldest warning, paid for in full (our damping prediction failed twice).
- **Pyrogenic niche construction.** [7,8] Fireproof seeds plus flammable expression wield fire against competitors — the eucalyptus syndrome, emergent from twenty template-sprouting rules.
- **Matching law / Roth-Erev / learning automata.** [9,10,11,12] The substrate’s natural learner is probability matching on reinforced propensities — the model behavioural game theory fits to humans, and the automaton Tsetlin built; MENACE’s beads, self-moving.
- **Proactive interference.** [13] Unlearning is seven times slower than learning: the old archive must decay before the new one wins.

- **Extinction burst.** [14] In the first window after a reversal the old behaviour *surges* (punishment attempts are attempts) before the cliff — behaviourism on schedule.
- **Cost of plasticity.** [15] Learning pays only when the environment outlives the relearning time; the crossover was measured, then moved by punishment — and it scales with memory size (a 1200-token registry flips in 15k events, a 110-token pouch in 5–10k): the bigger the brain, the stabler the world must be.
- **Fecundity beats information when errors are cheap.** A learner with actively wrong memory still out-breeds a random rival by sheer attempt mass — *r*-selection swamping accuracy selection; make errors lethal and the ordering inverts.
- **Adaptive value of learning.** [15,16,17] Selection over learning architectures favours punishment exactly where errors are lethal and frequent — evolved, not scored: the fitness function is the population itself.
- **Mutation–selection polymorphism.** [18] Architecture evolution settles into a stable mixture matching the seeded competition; differences below the mutational load blur; diversity, not a champion, is the ecosystem’s answer to nonstationarity.
- **Memory pays rent in land.** [19] Knowledge occupies substrate; its real-estate cost is a coexistence mechanism keeping the ignorant in the game — and, within one lineage, the space cost of an archive is a brake on its own expansion.

Two meta-observations. First, the falsification count: at least six explicit predictions made in these sessions were overturned by the measurements (bank damping, catalytic damping, catalytic retention merely surviving, the garden inheriting the bandit’s threshold, punishment helping at cheap-2k, the B=15k anomaly” that was succession). An instrument that only confirms is a mirror; this one pushed back. Second, the pedagogy: several of these laws were unknown to one of the authors before the substrate produced them, and obvious immediately after — a property we would claim for the substrate as a teaching device, not just a research one.

5 The main quantity: what an experiment costs

experiment	rules	runs	wall time	verdict
trait ladder	88	80	~2.5 h	oscillator, τ measured
seed bank (2 variants)	+31	72	~3 h	retention weight-set
catalytic readout	+20	18	~1.5 h	living memory, firestorm
bandit + punishment	25+2	66	~2.5 h	learning, economics
env/agent testbed	split	18	~0.5 h	bit-exact composition
garden + evolution	59–103	108	~3 h	selection over learning

Totals for the learning arc: roughly 350 exactly-accounted runs, some eight core-hours of compute, one long day of joint design time, zero engine changes. Median hypothesis-to-verdict: one to three hours, where the verdict includes the accounting, not a demo.

Against specialized native code the ledger is honest and mixed. On design time we win while the mechanism lies in the substrate’s native vocabulary (spatial, stochastic, populational): a new force is a small generator delta and inherits tracing, replay and verification for free, whereas a bespoke simulator pays for its dynamics *and* its instrumentation, per project. On runtime we lose, without excuses: the generic matcher runs ~2100 events per second where an incremental-rate kernel runs millions — three to five orders. At exploratory scale ($\leq 10^2$ runs of 10^5 – 10^6 events) the loss is noise and the substrate wins outright; at exponent-grade statistics the kernel wins decisively, and the right workflow is to use the substrate as the executable specification that a later kernel must reproduce bit-for-bit at small scale before it earns the large one.

And there is a deeper reason the trade is often right, which we owe to the gardener: **one engine fits all programs because the engine does not implement any model — it implements the meta-law the**

models share. Weighted local stochastic rewriting is the common physics beneath contact processes, forest fires, sandpiles, foraging, dormancy and reinforcement; a specialized simulator reimplements nature once per experiment, while Zahradnice amortizes nature and lets each program be merely a particular world. The orders of magnitude are the price of running directly on the meta-law; the catalogue of Section 5 is what that price buys.

6 Honest limits

- Reproducing known laws validates the *instrument*, not new science; nothing here is a novel result about ecology or learning — the novel-result track (parallel batching shifting critical points) is a separate, narrower paper.
- The ledger measures problems the substrate is good at; counters, rigid geometry and global conditions are expensive here (earlier sessions paid that bill), and the selection of experiments reflects it.
- Parameters are hand-tuned, one to two smoke iterations each; no optimality claims. Thresholds are bracketed, not resolved.
- The cheap-garden learner advantage is largely fecundity, not intelligence (documented in place); attempt mass and learning are entangled by catalysis.
- All fields are small (33×64) and budgets finite; absorbing states and metastability are facts of the machine we exploit, not eliminated nuisances.
- Design and verification were done by one human—one model pair; independent replication is a matter of running the committed scripts (all runs are seed-deterministic).

7 Where this leaves the project

The founding question of the research chapter — can reward-coupled adaptation live entirely in state, with immutable rules and no optimizer anywhere — is answered affirmatively, constructively, and cheaply, with its own textbook attached. Deliberately parked, with reasons recorded in the repository notes: contextual token species (state-indexed policies), coarse coding (tile-coded features), delayed reward via eligibility flowers, the reflective layer (rules as matter), and the pixel-scale game branch. The narrow archival paper on conflict-serialized parallelism and critical behaviour proceeds independently.

8 References

1. van der Pol, B. (1926). On relaxation-oscillations. *Phil. Mag.* 2, 978–992.
2. Tilman, D. (1994). Competition and biodiversity in spatially structured habitats. *Ecology* 75, 2–16.
3. Cohen, D. (1966). Optimizing reproduction in a randomly varying environment. *J. Theor. Biol.* 12, 119–129.
4. Hairston, N. G. et al. (1999). Rapid evolution revealed by dormant eggs. *Nature* 401, 446.
5. Little, J. D. C. (1961). A proof for the queuing formula $L = \lambda W$. *Operations Research* 9, 383–387.
6. Åström, K. J., Murray, R. M. (2008). *Feedback Systems*. Princeton University Press.
7. Mutch, R. W. (1970). Wildland fires and ecosystems — a hypothesis. *Ecology* 51, 1046–1051.
8. Bond, W. J., Midgley, J. J. (1995). Kill thy neighbour: an individualistic argument for the evolution of flammability. *Oikos* 73, 79–85.
9. Herrnstein, R. J. (1961). Relative and absolute strength of response as a function of frequency of reinforcement. *J. Exp. Anal. Behav.* 4, 267–272.
10. Roth, A. E., Erev, I. (1995). Learning in extensive-form games. *Games and Economic Behavior* 8, 164–212.
11. Narendra, K. S., Thathachar, M. A. L. (1989). *Learning Automata: An Introduction*. Prentice-Hall. (Tsetlin, M. L., 1973, *Automaton Theory and Modeling of Biological Systems*.)
12. Michie, D. (1963). Experiments on the mechanization of game-learning. Part I. *The Computer Journal* 6, 232–236. (Gardner, M., 1962, Mathematical Games, *Sci. Am.* 206(3).)
13. Underwood, B. J. (1957). Interference and forgetting. *Psychol. Review* 64, 49–60.

14. Lerman, D. C., Iwata, B. A. (1995). Prevalence of the extinction burst and its attenuation during treatment. *J. Appl. Behav. Anal.* 28, 93–94. (Skinner, B. F., 1938, *The Behavior of Organisms*.)
15. Stephens, D. W. (1991). Change, regularity, and value in the evolution of animal learning. *Behavioral Ecology* 2, 77–89.
16. Dunlap, A. S., Stephens, D. W. (2009). Components of change in the evolution of learning and unlearned preference. *Proc. R. Soc. B* 276, 3201–3208.
17. Dunlap, A. S., Stephens, D. W. (2014). Experimental evolution of prepared learning. *PNAS* 111, 11750–11755.
18. Crow, J. F., Kimura, M. (1970). *An Introduction to Population Genetics Theory*. Harper & Row.
19. Chesson, P. (2000). Mechanisms of maintenance of species diversity. *Annu. Rev. Ecol. Syst.* 31, 343–366.

Reproduce it. Clone the repository, make `zahradnice-size`, and run any experiment’s sweep script; every number above regenerates from a seed. Or simply watch: `./zahradnice experiments/garden/garden.cfg` — white forgets, yellow remembers, cyan forgets on purpose; hold `b` to change the season under their feet.